



软件体系结构

第1章 概述

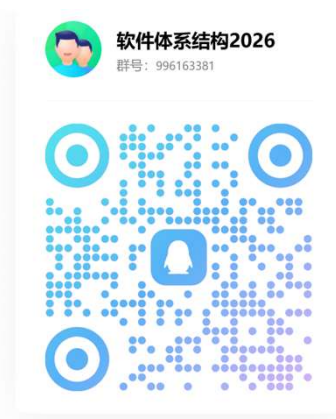
赵庆玲

南京理工大学计算机科学与工程学院

ada_zhao@njust.edu.cn

- 授课人：
 - 赵庆玲
 - ada_zhao@njust.edu.cn
- 考核：
 - 平时+考勤 10%
 - 期末考试（半开卷） 60%
 - 软件大作业 30%

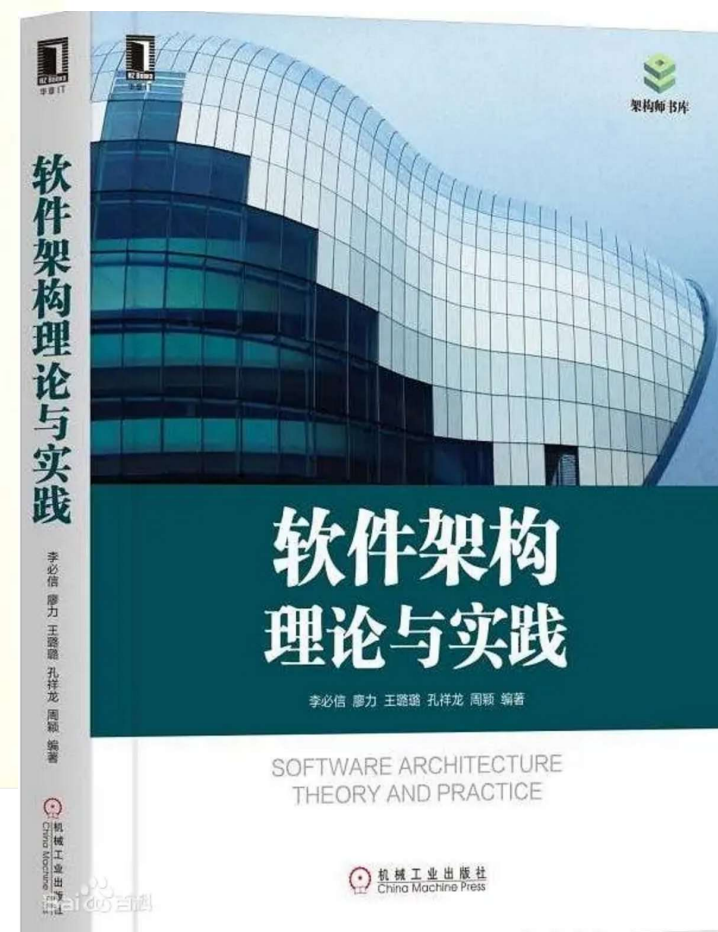
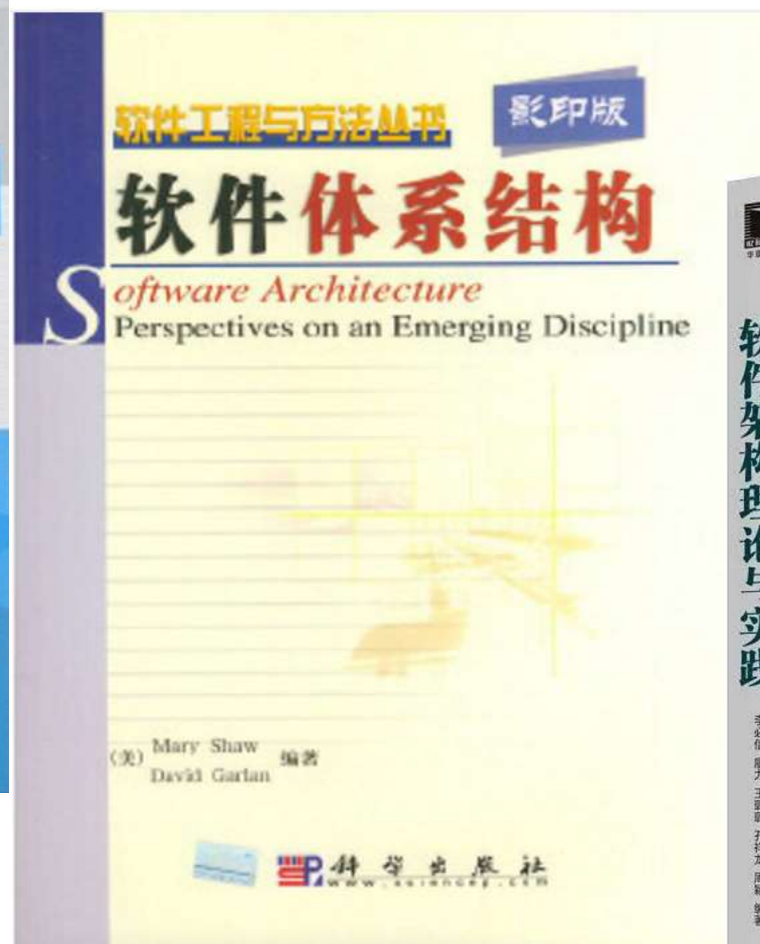
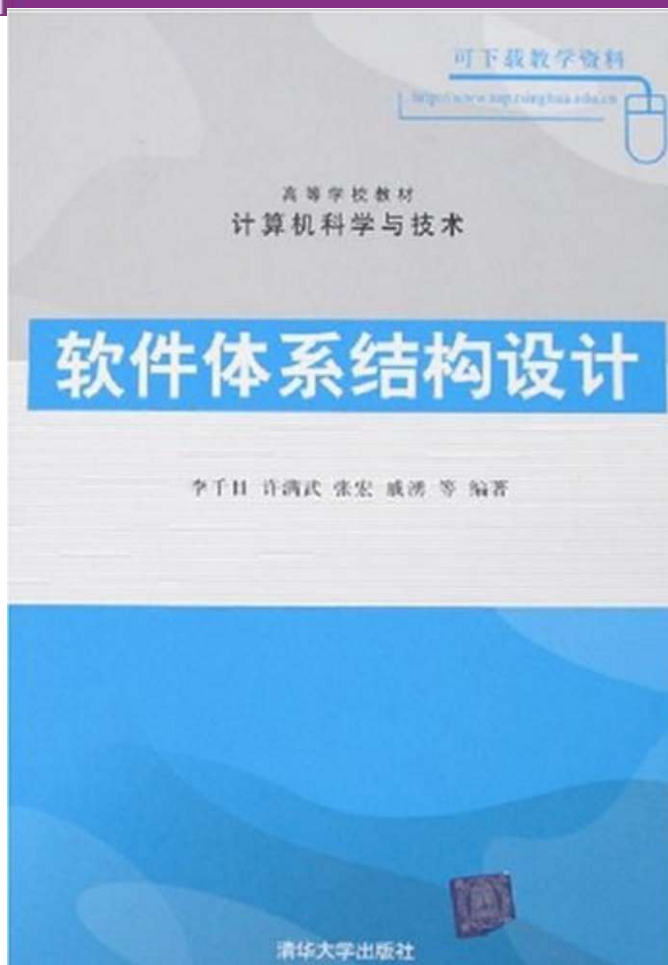
- QQ 群:



教材



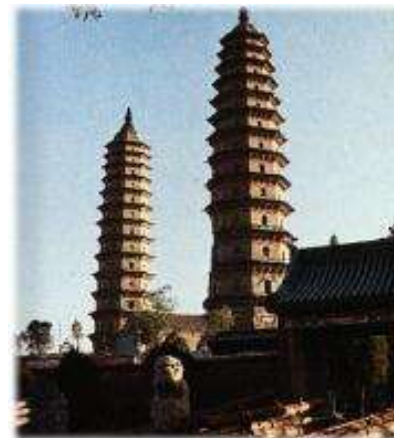
南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY



“建筑体系结构”



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY



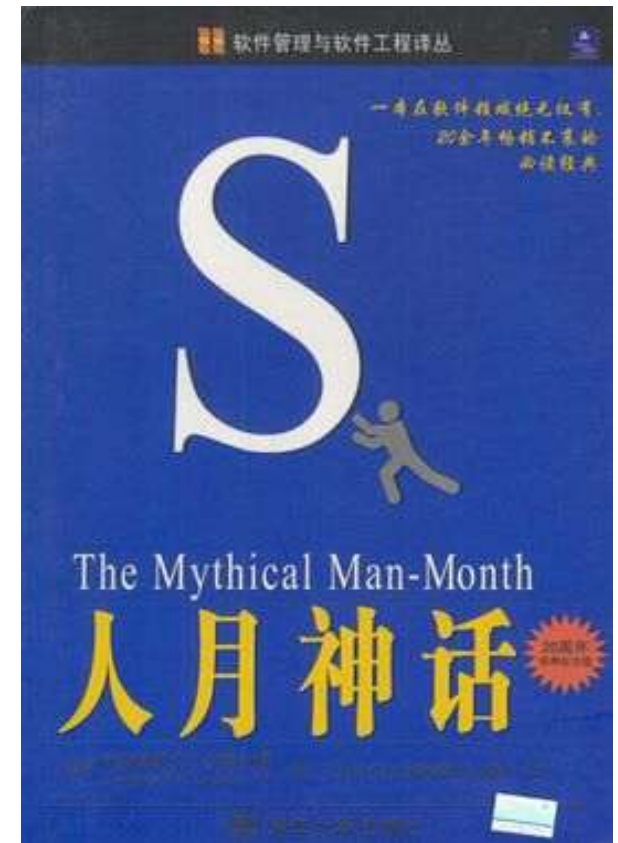
The causes of the software crisis

软件危机的原因

- The scale of the software is getting bigger and bigger
- 软件的规模越来越大
- Increasing complexity
- 日益复杂
- Delivery time is relatively short
- 交货时间比较短

- IBM 360 OS, 5000多人年, 耗时4年(1963—1966), 花费2亿多美元, 每次发布发现1000多个bug
- **The IBM 360** operating system developed by IBM of the United States **from 1963 to 1966**, which took **5000 people** a year to work, with at most **1000 people** working on development, and nearly a million lines of programming...According to statistics, each new release of the operating system is the result of finding **1000 bug** fixes in the previous version...

- The director of the project, F. D. **Brooks**, later summed up his painful lessons in the development process: ‘... *Just as a fugitive beast falls into the mire and becomes a moribund struggle, the more it struggles, the deeper it sinks, the last it cannot escape. Programming is just like this. A lot of programmers are forced to struggle in the mud... No one expected that the problem would fall into such a predicament...*’. The history of the IBM360 operating system has become a classic example of software development projects.



The crisis manifested itself in several ways

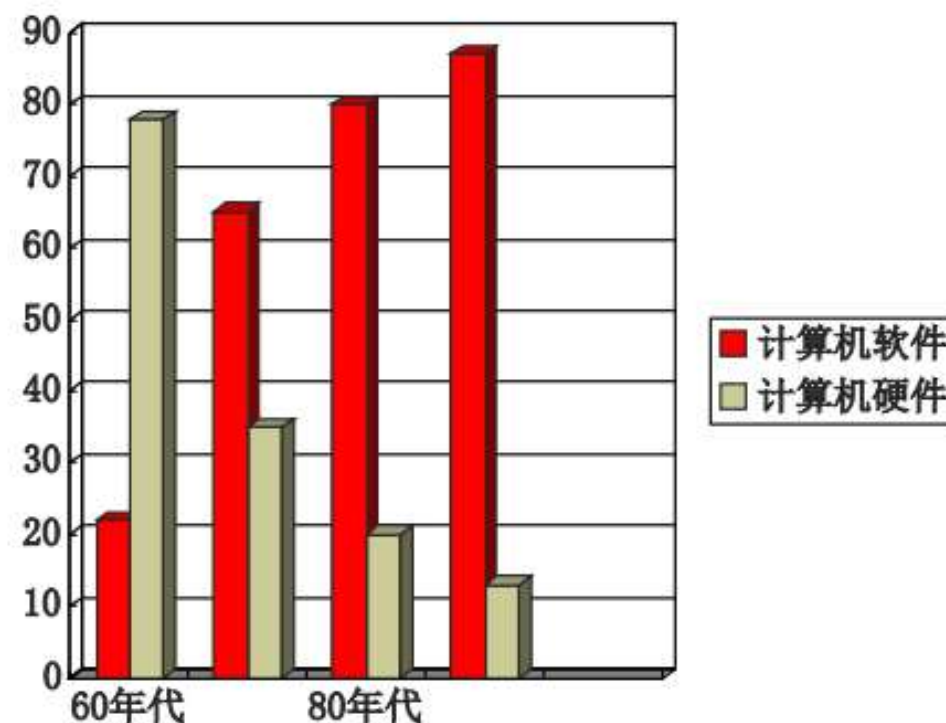
危机表现在几个方面

- Increasing software costs
不断增加的软件成本
- Development progress is difficult to control
开发进程难以控制
- Software was very inefficient
软件效率很低
- Software was of low quality
软件质量低劣
- Projects were unmanageable and code difficult to maintain
项目无法管理，代码难以维护

软件危机的表现



- 美国空军：1955年软件占总费用(计算机系统)的18%，70年60%，85年达到85%
- 美国全球军事指挥控制系统，硬件1亿美元，软件高达7.2亿美元



- Development progress is difficult to control
 - In the software development process, user's demand changes and other unexpected situations emerge in an endless stream, making the software development process is difficult to ensure that the plan is achieved on schedule.

在软件开发过程中，用户的需求变化和其他不可预见的情况层出不穷，使得软件开发过程难以保证计划按期完成。

- Blindly increasing the software developer is not proportional to improve the ability of software development. On the contrary, with the increase of the number of personnel, the problems of personnel organization, coordination, communication, training and management will be even more serious.

盲目增加软件开发人员与提高软件开发能力不成正比。相反，随着人员数量的增加，人员组织、协调、沟通、培训和管理的问题将更加严重。

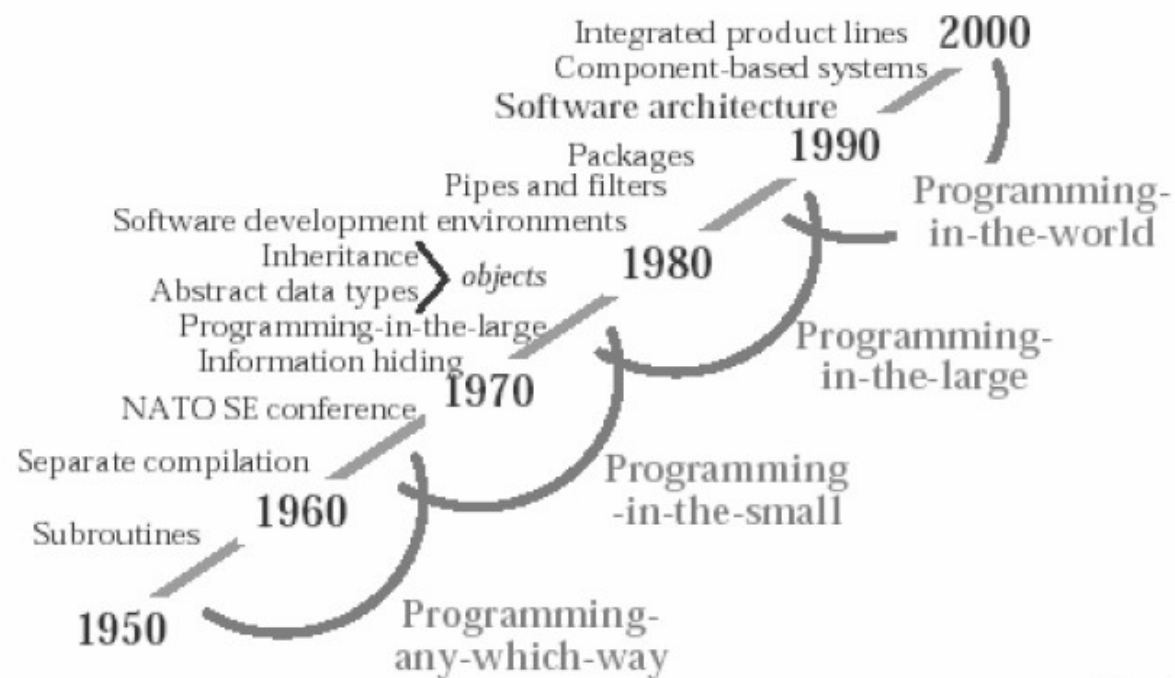
- Software was very inefficient and of low quality
 - Due to the lack of guidance of engineering thought, programmers almost always use their own ideas to replace the user's demand for software. The software designs with randomness, many functions are just a programmer's wishful thinking, which is the important factor of the software is unsatisfactory
 - 由于缺乏工程思想的指导，程序员几乎总是用自己的思想来代替用户对软件的需求。软件设计具有随机性，许多功能只是一个程序员的一厢情愿，这是软件不满意的重要因素。

软件危机的表现



- **Projects were unmanageable and code difficult to maintain**
 - Because in the process of software design and development, the developers didn't strictly follow the software development standard, and there is no complete record of the true state of the system documentation, resulting in great difficulties in software maintenance.
 - 因为在软件设计和开发过程中，开发人员没有严格遵守软件开发标准，没有完整的系统文档真实状态记录，导致软件维护难度很大。
 - Especially in the process of using the software, the original developer may have left the original development organization for various reasons, making the software almost unmaintainable
 - 特别是在软件的使用过程中，原来的开发人员可能由于各种原因离开了原来的开发组织，使软件几乎无法维护。
 - Data show that the industry is responsible for the maintenance of software to pay for the total hardware and software costs of 40-75%
数据显示，该行业负责维护软件，支付硬件和软件总成本的40-75%

Antecedents of Software Architecture



- 关于犬舍、建筑和建模的故事 ...
 - We are building a software skyscraper in a Kennel's way
 - 我们正在用狗舍的方式建造一座软件摩天大楼



犬舍的体系结构设计

- 一个人建造
- 需要
 - 在建造者自己脑中设计
 - 简单的建造过程
 - 简单的工具



独栋房屋的体系结构设计



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 团队建造，要求高效、可靠
- 需要
 - 设计、建模
 - 明确定义的建造过程
 - 强大的工具

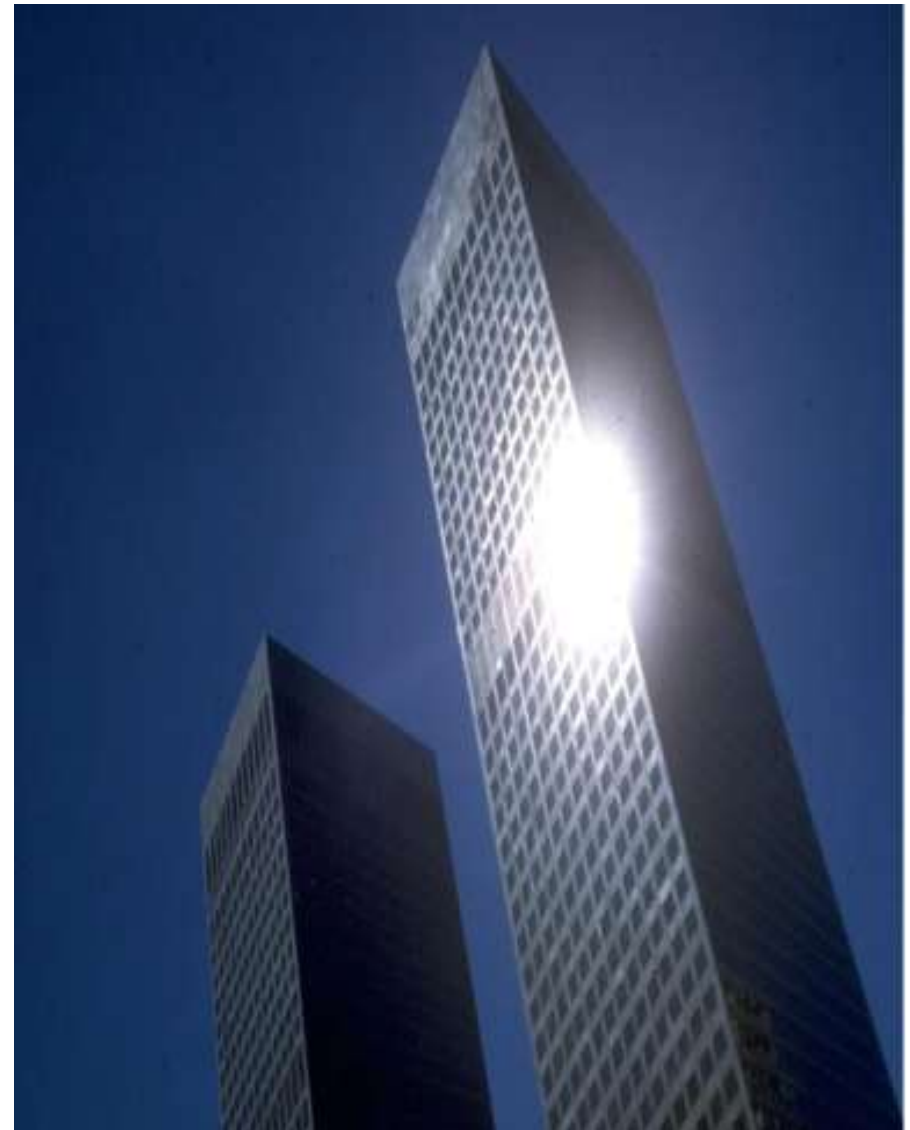


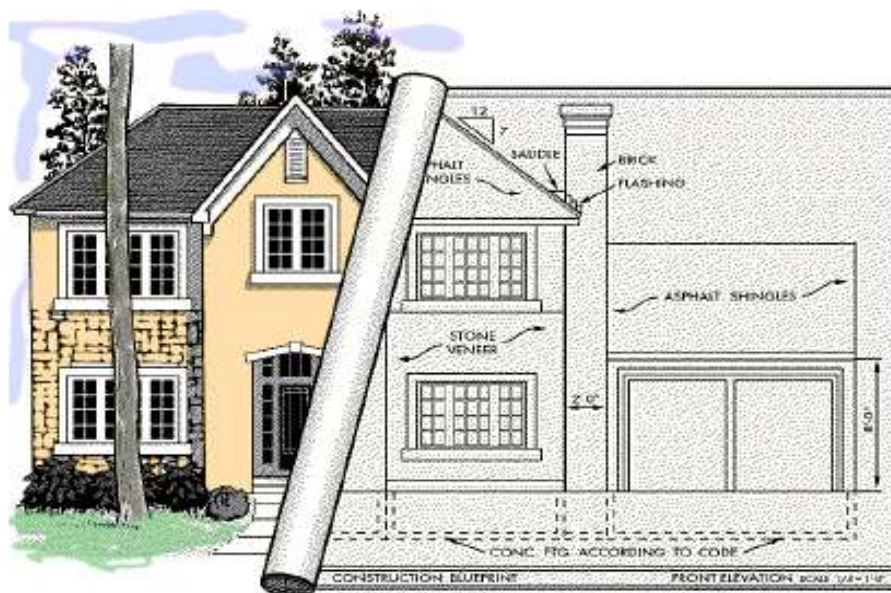
摩天大楼的体系结构设计



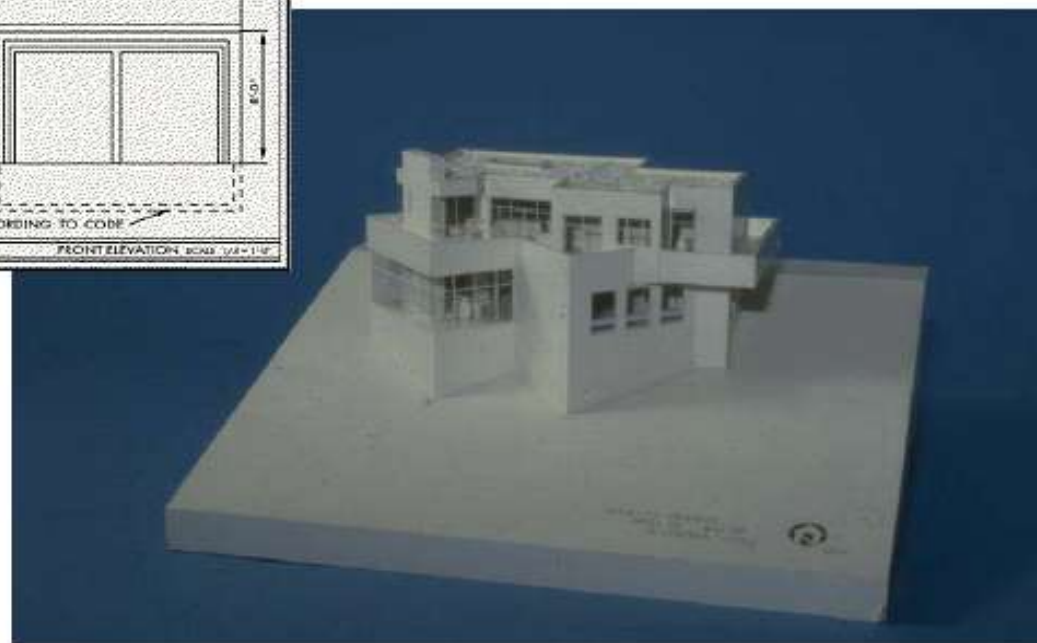
南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

如何建造？





Point 1: the part and the whole



Point 2: Modeling and Implementation



- “计算机体系结构” 定义是1964年C. M. Amdahl在介绍IBM 360系统时提出的：计算机体系结构是程序员所看到的计算机的属性，即概念性结构与功能特性。
- “软件体系结构” 的概念是20世纪90年代中期David Garlan和Mary Shaw在《An Introduction to Software Architecture》中提到的。

<http://www.cs.utexas.edu/users/EWD/>

<http://www.cs.utexas.edu/users/EWD/ewd01xx/EWD196.PDF>

- 软件体系结构的产生
 - **20世纪90年代**， 研究人员展开了关于软件体系结构的基础研究，主要集中在体系结构风格(模式)、体系结构描述语言、体系结构文档和形式化方法。
 - 软件体系结构在高级层次上对软件进行描述，便于软件开发过程中各个视角（例如用户、业务和系统）的统一，能够及早发现开发中的问题并支持各种解决方案的评估和预测。
 - 其意义贯穿软件生命周期的各个阶段。

软件体系结构的发展阶段



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 基础研究阶段（1968-1994）
- 概念体系和核心技术形成阶段（1991-2000）
- 理论体系丰富发展阶段（1996-1999）
- 理论完善和普及应用阶段（1999年至今）

软件体系结构的发展阶段



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 1. 基础研究阶段（1968-1994）
 - 1968北大西洋公约组织(NATO)会议上第一次出现词语“软件体系结构”
 - 直到20世纪80年代，“体系结构”一词大部分情况下被用于表示计算机系统的物理结构，偶尔被用于表示计算指令集的特定体系。
 - 随着软件规模增大，开发者已经开始尝试模块化（modularization）的实践

- 1. 基础研究阶段（1968-1994）
 - 模块化:指的是一种软件开发方法，把一个待开发的软件分解成若干小的简单的部分，称为模块。每一个模块都独立地开发、测试，最后再组装出整个软件。
 - 模块化规则：
 - 高内聚，低耦合
 - 模块大小适度
 - 模块调用链的深度 (嵌套层次)不可过多
 - 接口干净，信息隐蔽
 - 尽可能地复用已有模块

- 2.概念体系和核心技术形成阶段（1991-2000）
 - 1991年，Winston W. Royce 与 Walker Royce 首次对软件体系结构进行了定义。
 - 1992年D E.Perry与A L.Wolf对软件体系结构进行了阐述，提出了著名的{elements, forms, rationale} = software architecture公式
 - 1996年CMU/SEI的Mary Shaw和David Garlan，出版了《Software Architecture: Perspectives on an Emerging Discipline》，对软件体系结构概念的内涵与外延进行了详尽阐述，对软件体系结构概念的形成起到了至关重要的作用。

软件体系结构的发展阶段



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 2.概念体系和核心技术形成阶段（1991-2000）
 - 从1995年起，软件体系结构研究领域开始进入快速发展阶段
 - Booch, Rumbaugh and Jacobson从另一个角度对软件体系结构的概念进行了全新的诠释，认为体系结构是一系列重要决策的集合。
 - 提出了软件体系结构实践方法体系SAAM；
 - Rechtin 与 Mark Maier在1998年出版的《The Art of Systems Architecting》很好地阐述了系统与软件的关系。

- 2.概念体系和核心技术形成阶段（1991-2000）

- 这一阶段最重要的成果之一就是软件组件（component）技术
- 组件化开发并不等同于模块化开发。模块化开发只是在逻辑上做了切分，物理上（开发出的系统代码）通常并没有真正意义上的隔离。
- 组件化也不等同于应用集成，应用集成是将一些基于不同平台或不同方案的应用软件和系统有机地集成到一个无缝的、并列的、易于访问的单一系统中，建立一个统一的综合应用。
- 组件化应比模块化更独立，但比应用集成结合得更加紧密。

- 3.理论体系丰富发展阶段（1996-1999）

- 随着基于组件软件体系结构理论的建立，与之相关的一些研究方向逐渐成为软件工程领域的研究热点。主要研究方向包括：
 - 软件体系结构的描述与表示；
 - 软件体系结构分析、设计与测试；
 - 软件体系结构发现、演化与重用；
 - 基于软件体系结构的开发方法；
 - 软件体系结构的风格
 - 等等

- 4.理论完善和普及应用阶段（1999-至今）
 - 1999年，第一届IFIP软件体系结构会议召开；Open Group 提出了体系结构描述Markup语言，一种基于XML的体系结构描述语言，支持广泛的体系结构模型共享；软件产品线成为了软件体系结构的一个重要分支，吸引了大量的大型企业的关注。
 - 2000年，IEEE 1471-2000发布，为软件体系结构的普及应用制定了标准化规范，该标准随后分别于2007年与2011年得到扩充与修改。
 - 2003年，《Software Architecture in Practice》一书出版。

软件体系结构是一个软件系统的高层结构，它高度抽象，超越了算法和数据结构，基本着眼点是系统结构和需求与实现之间的交互，是一个用于理解系统级目标的框架。

Mary Shaw和David Garlan认为，软件体系结构是软件设计过程中，超越计算中的**算法设计和数据结构设计**的一个层次。体系结构问题包括各方面的组织和全局控制结构，通信协议同步，数据存取，给设计元素分配特定功能，设计元素的组织，规模和性能，在各设计方案间进行选择等。

软件体系结构对软件生命周期中的各个阶段以及项目的管理有着重要的影响。

- 体系结构设计是基于需求分析的一个迭代的过程，这就不可避免地会与需求分析阶段进行交互。体系结构的设计又是后续阶段详细设计的基础。
- 在开发阶段，开发人员正确地理解系统的体系结构是开发工作顺利进行的前提。
- 在测试阶段，体系结构对测试起指导作用。
- 在维护阶段，维护中的大量时间都花费在对现存代码的理解上，如果原始的设计结构能够得到清楚和明确的表达，特别是高层次的表达，就可以大大减少花在这方面的时间，而如果不知道系统的体系结构则使维护工作很难进行。对于一些已经存在但是不知其体系结构的系统，甚至有必要进行体系结构重构。
- 另外，体系结构对于项目的组织管理也具有重要的意义，合理的体系结构设计还有利于开发任务在开发人员之间的分配，有利于开发人员之间关系的协调。

综上所述，软件体系结构是整个软件设计成功的基础和关键所在：

- 软件体系结构是软件开发中各种角色之间进行交流的手段
- 软件体系结构是早期设计决策的体现
- 软件体系结构制约着软件的质量属性
- 软件体系结构是可重用的模型
- 软件体系结构影响着开发和维护组织的组织结构
- 软件体系结构为软件系统的整个开发过程都提供了有力的支持

1、软件体系结构研究的主要内容

- 软件体系结构描述语言
- 体系结构描述构造与表示
- 体系结构的设计、分析与验证
- 体系结构发现、演化与重用
- 基于体系结构的软件开发方法
- 特定领域的软件体系结构
- 软件体系结构支持工具
- 软件产品线体系结构

(1) 软件体系结构描述语言



软件体系结构描述语言（Architecture Description Language，简称为ADL）是一种形式化语言，它在底层语义模型的支持下，为软件系统的概念体系结构建模提供了具体的语法和概念框架。基于底层语义的工具为软件体系结构的表示、分析、演化、细化、设计过程等提供支持，包括构件、连接件和约束三个基本元素。主要的体系结构描述语言有C2，UniCon，MetaH，Aesop，SADL，Rapide，Wright等。ADL的目的就是提供一种规范化的体系结构描述，以便与人交流，并能够用提供的工具对许多实例进行分析。

(2) 体系结构描述构造与表示



按照一定的描述方法，用体系结构描述语言对体系结构进行说明的结果则称为体系结构的表示，而将描述体系结构的过程称为体系结构构造。在体系结构描述方面，Kruchten提出的“4+1”模型是当今软件体系结构描述的一个经典范例，该模型由逻辑视图、开发视图、过程视图和物理视图组成，并通过场景将这四种视图有机地结合起来，比较细致地描述了需求和体系结构之间的关系。而Booch从UML的角度出发给出了一种由设计视图、过程视图、实现视图和部署视图，再加上一个用例视图构成的体系结构描述模型。IEEE于1995年成立了体系结构工作组，综合了体系结构描述研究的成果，并参考业界的体系结构描述的实践，起草了体系结构描述标准IEEE P1471。

(3) 体系结构的设计、分析与验证



生成一个**满足软件需求的体系结构**的过程即为体系结构设计。体系结构设计的本质在于：将系统分解成相应的组成成分，并将这些成分重新组装成一个系统。软件体系结构设计是软件设计中非常重要的一个环节，软件开发过程中只要需求和体系结构确定之后，这个软件基本上也就定型了。体系结构设计有**过程驱动方法**和**问题列表驱动方法**。体系结构设计研究的重点内容之一是**体系结构风格**，体系结构风格在本质上反映了一些特定的元素按照特定的方式组成一个特定的结构，该结构应有利于上下文环境中的特定问题的解决。体系结构是对系统的高层抽象，并只对感兴趣的属性进行建模。由于体系结构是在软件开发过程之初产生的，因此好的体系结构可以减少和避免软件错误的产生和维护阶段的高昂代价。体系结构是系统集成的蓝本、系统验收的依据，体系结构本身需要分析与测试，以确定这样的体系结构是否满足需求。比较重要的体系结构分析（评估）方法有**体系结构权衡分析方法**

（Architecture Tradeoff Analysis Method，简称为ATAM）、**软件体系结构分析方法**（Software Architecture Analysis Method，简称为SAAM）和**中间设计积极评审法**（Active Reviews for Intermediate Design，简称为ARID）。

(4) 体系结构发现、演化与重用

对于一个已经存在但是不知道其体系结构的软件系统，对其进行维护是一件很困难的事情，这时可以通过**体系结构发现**来解决这个问题。体系结构发现就是从已经存在的软件系统中提取出软件体系结构，这可以帮助维护人员理解系统，保证维护工作的顺利进行。

由于系统需求、技术、环境、分布等因素的变化而最终导致软件体系结构的变动，称为**软件体系结构演化**。而**体系结构重用**是属于比代码重用更高级的设计重用。重用是软件工程领域所倡导的有效技术之一，体系结构级的重用会导致大批构件的重用，不仅极大地提高了软件开发的**效率**、降低了软件开发的**成本**，而且对于提高多种软件**质量**都有很大的作用。

(5) 基于体系结构的软件开发方法



在本质上，软件体系结构是对软件需求的一种抽象解决方案。在引入了体系结构的软件开发之后，应用系统的构造过程变为

“问题定义——软件需求——**软件体系结构**——软件实现”，

可以认为软件体系结构架起了软件需求和软件设计之间的一座桥梁。在基于构件和软件体系结构的软件开发逐渐成为主流的开发方法的情况下，已经出现了基于构件的软件工程。



(6) 特定领域的软件体系结构

特定领域的软件体系结构（Domain Specific Software Architecture，简称为**DSSA**）是一门以**软件重用**为核心，研究软件应用框架的获取、表示和应用等问题的软件方法学。**DSSA**将软件体系结构理论应用到具体领域，因为特定领域的应用具有相似的特征，所以可以借鉴领域中已经成熟的软件体系结构。通过特定领域的软件体系结构，可以实现解决方法在某个领域内的重用。

(7) 软件体系结构支持工具



几乎每一种软件体系结构描述语言都有相应的支持工具，如 Unicon，Aesop 等体系结构支持环境，C2 的支持环境 ArchStudio 等。除了软件体系结构描述语言的支持工具外，还有一些基于软件体系结构的开发方法的支持工具，例如 ABC（architecture-based component composition）是一种基于软件体系结构面向构件的软件开发方法，ABC-Tool 就是 ABC 方法的支持工具。另外还有一些支持体系结构静态分析的工具、体系结构性能仿真工具等，但是与其他成熟的软件开发环境相比，体系结构的支持工具还不是很实用。

(8) 软件产品线体系结构



软件体系结构的开发是大型软件系统开发的关键环节，对于软件产品线的开发来说更是具有至关重要的作用。在这种开发生产中，基于同一个软件体系结构，可以创建具有不同功能的多个系统。在软件产品族之间共享体系结构和一组可重用的构件，可以降低开发和维护的成本。

2 SA的定义



- “There is no standard, universally-accepted definition of the term, for software architecture is a field in its infancy, although its roots run deep in software engineering.”

<http://www.sei.cmu.edu/architecture/definitions.html>

软件体系结构尚处在发展期，对于其定义，目前学术界尚未形成统一意见，不同学者有不同看法。

- 软件体系结构的定义一出现就存在比较大的争论。
 - 研究人员一般认为：软件体系结构就是一个系统的草图。
 - 业界人士虽然也认同研究人员对软件体系结构概念的描述，但实际上很多时候很难用某个软件体系结构定义来对问题进行系统的、全面的、准确的刻画，也很难解决到满意的程度。
 - 由于学术界和工业界的联系不紧密，导致对软件体系结构的认识不一致，使得软件体系结构至今很难有个统一的定义。

- 软件体系结构的定义驳杂多端，其中影响较大的定义派别是：**组成派和决策派**。
- **组成派**关注于软件本身，将软件体系结构看做构件和交互的集合；
- **决策派**关注于软件体系结构中的实体（人），将软件体系结构视为一系列重要设计决策的集合。

- 组成派定义依据：软件体系结构主要反映系统由哪些部分组成，以及这些部分是如何组成的，强调软件系统的整体结构和配置。

- Dewayne Perry and Alexander Wolf, 1992:

A set of architectural (or, if you will, design) elements that have a particular form. Perry and Wolf distinguish between processing elements, data elements, and connecting elements, and this taxonomy by and large persists through most other definitions and approaches.

软件体系结构是一组具有特定形式的体系结构元素(elements)。这组元素分为3类：负责完成数据加工的处理元素(processing elements)、作为被加工的信息的数据元素(data elements)和连接元素(connecting elements)。连接元素用于把体系结构的不同部分组合连接到一起。

定义1: (Perry & Wolf模型)

软件体系结构={元素, 形式, 准则}。

- 其中, 元素是指具有一定形式的结构化元素。包括处理元素 (processing elements)、数据元素 (data elements) 和连接元素 (connecting elements)。
- 体系结构组成由加权的属性 (weighted properties) 和关系 (weighted relationships) 构成。属性用来约束体系结构元素的选择, 关系用来约束体系结构元素的放置 (placement)。
- 体系结构准则捕获在选择体系结构风格、体系结构元素和体系结构形式的选择动机。

- Garlan and Shaw, 1993:

*...beyond the algorithms and data structures of the computation; designing and specifying the **overall system structure** emerges as a new kind of problem. Structural issues include **gross organization and global control structure; protocols for communication, synchronization, and data access; assignment of functionality to design elements; physical distribution; composition of design elements; scaling and performance; and selection among design alternatives.***

Mary Shaw和David Garlan认为，软件体系结构是软件设计过程中，超越计算中的算法设计和数据结构设计的一个层次。体系结构问题包括各方面的组织和全局控制结构，通信协议同步，数据存取，给设计元素分配特定功能，设计元素的组织，规模和性能，在各设计方案间进行选择等。

定义2: (Garlan & Shaw 模型)

软件体系结构={构件, 连接件, 约束}。

其中:

- 1) 构件(component) : 可以是一组代码, 如程序的模块; 也可以是一个独立的程序, 如数据库服务器。构件是相关对象的集合, 运行后实现某计算逻辑。它们或是结构相关(如嵌套对象, 其中的被嵌对象是嵌套对象的一部分)或是逻辑相关(若干聚集对象完成某功能)。

构件相对独立, 仅通过接口与外部相互作用, 可作为独立单元嵌入到不同应用系统中。构件的定制和规范化对于实现构件的复用有重要意义。

定义2: (Garlan & Shaw 模型)

软件体系结构={构件, 连接件, 约束}。

其中:

- 2) 连接件(connector): 可以是过程调用、管道、远程过程调用等, 用于表示构件之间的相互作用。连接件是构件的粘合剂, 它把不同的构件连接起来构成体系结构的一部分。连接件也是一组对象。它一般表现为框架式对象或转换式对象(调用远程构件资源), 例如“桩”, “代理”对象等。

定义2: (Garlan & Shaw 模型)

软件体系结构 = {构件, 连接件, 约束}。

其中:

- 3) 约束(constraint): 约束一般为对象连接时的规则, 或指明构件连接的势态和条件。例如, 上层构件可要求下层构件的服务, 反之不行; 两对象不得递归地发消息; 代码复制迁移的一致性约束; 在什么条件下此种连接无效等。

在第一届软件系统体系结构国际讨论会（94年）上，Mary Shaw对于当时术语使用的混乱情况予以了澄清。从会议论文中，Shaw整理出他们的定义和观点，并对当时的各种观点作了如下的分类：

结构模型； 框架模型； 动态模型； 过程模型。

(1)结构模型

结构模型认为，软件体系结构由构件、构件之间的连接和一些其他方面组成。这些方面包括如下几类：

- 配置，风格。
- 约束，语义。
- 分析，属性。
- 原理，需求。

(2)框架模型

框架模型的观点与结构模型相似，但其重点在于整个系统的连贯结构(这种结构通常是唯一的)，这与重视其组成恰好相反。框架模型常常以某种特定领域或某类问题为目标。

(3) 动态模型

动态模型强调系统的行为质量。“动态”可以有多种含意。它可以是指整个系统配置的变化也可以是指禁止预先激活了的通信或交互，还可以是指计算中表现中的动态特性，如改变数据的值。

(4) 过程模型

过程模型关注系统结构的构建及其步骤和过程。在这一观点下，体系结构是所进行的一系列过程的结果。

定义3: (Bass定义)

Bass等人在《Software Architecture in Practice》一书中提到，程序或计算系统的软件体系结构是系统的一个或多个结构，包括软件构件(components)、构件的外部可视属性(properties)和构件之间的关系。

这里的外部可视属性，是指其他构件认为该构件所具备的特征，如所提供的服务、具有的性能特点、错误处理机制、共享资源的用法等。要注意的是，此定义中，特意未指明什么是构件，什么是关系。构件既可以是对象，也可以是进程，还可以是函数库，或是数据库。

定义4: ([ISO/IEC/IEEE 42010](#))

2011年ISO/IEC/IEEE标准中定义软件体系结构是某一系统的基本组织结构，其内容包括软件构件，构件间的联系，构件与其环境间的联系，以及指导上述内容设计与演化的原理。

定义5: Boehm模型

软件体系结构 = {构件, 连接件, 约束, 不同人员的需求, 准则}。

Barry Boehm和他的学生提出, 软件体系结构包括系统构件、连接件、约束的集合, 反应不同人员需求的集合, 以及准则的集合。其中, 这些准则能够说明由构件、连接件和约束所定义的系统在实现时是如何满足系统不同人员需求的。

- **决策派定义依据：软件体系结构设计是软件设计的一部分，软件设计实际上是开发人员意志和决策在软件开发过程中的体现，软件体系结构更是高层领导和体系结构师意志和决策的体现，强调的是设计决策，所以更加注重体系结构风格和模式的选择。**

- Booch, Rumbaugh, and Jacobson, 1999:

An architecture is the set of significant decisions about the **organization of a software system**, the selection of the structural elements and their interfaces by which the system is composed, together with their behavior as specified in the collaborations among those elements, the **composition of these structural and behavioral elements** into progressively larger **subsystems**, and the **architectural style** that guides this organization---these elements and their interfaces, their collaborations, and their composition (The UML Modeling Language User Guide, Addison-Wesley, 1999).

- 体系结构是一系列重要决策的集合，这些决策与以下内容相关：软件的组织，构成系统的结构元素及其接口的选择，这些元素在相互协作中明确表现出的行为，这些结构元素和行为元素进一步组合构成的更大规模的子系统，和引导这一组织——包括这些元素及其接口、它们的协作、它们的组合——的体系结构风格。

定义6: (Booch & Rumbaugh & Jacobson 定义)

软件体系结构 = {组织, 元素, 子系统, 风格}

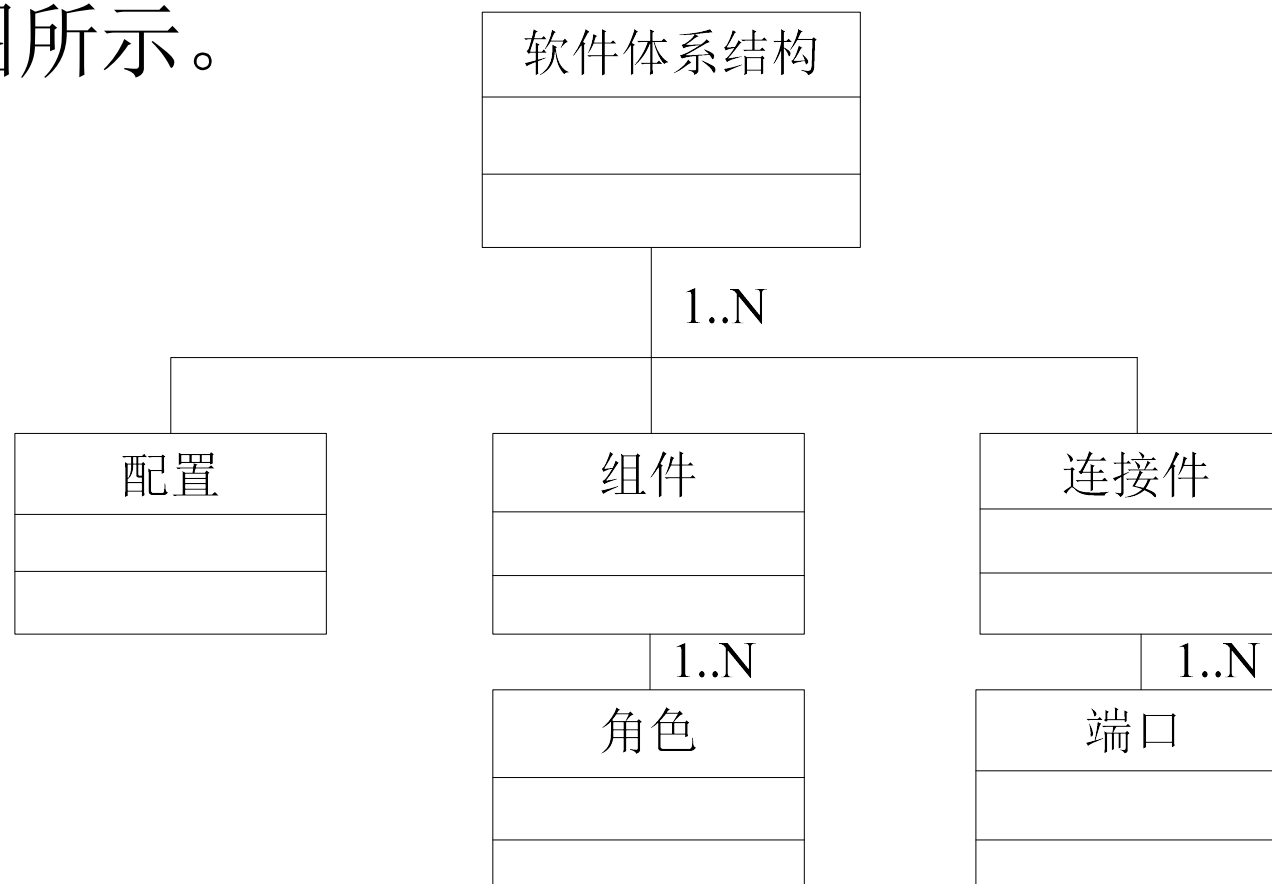


- 2005年 Anton等
 - 软件体系结构是体系结构层次上所有设计决策的集合体，这些设计决策与以下内容有关：体系结构改造的影响、原理、设计准则、设计约束以及附加需求。
- 2006年 Kruchten等
 - 设计决策+设计，这里的设计指的是设计决策的推理过程。

- Vivek Khare认为软件体系结构是设计和构建软件应用的科学和艺术，这些软件应用满足生命周期中用户的各种需求；
- Aakash Ahmad认为软件体系结构是包含设计、演化、构建配置的推理和构件互连关系的高层抽象结构；
- Andreas Rausch认为软件体系结构是一个针对软件改变的框架；
- Muthu Rajagopal认为软件体系结构是能够有效组合在一起的软件和硬件构件的集合，这些构件组合后能满足预期需求。

- **基于国内外普遍认可的看法推荐如下的参考定义框架：**
 - **软件体系结构包括构件、连接件和约束三种基本元素，以及端口和角色两种元素。**
 - **构件作为一个封装的实体，仅通过其接口结构与外部环境交互，而构件的接口由一组端口组成，每个端口表示了构件和和外部环境的交互点。**
 - **连接件的接口由一组角色组成，每个角色定义了相关交互的参与者。**

- 软件体系结构一般由如下五种元素构成：组件（component），连接件（connector），配置（configuration），端口（port）和角色（role）。如下图所示。



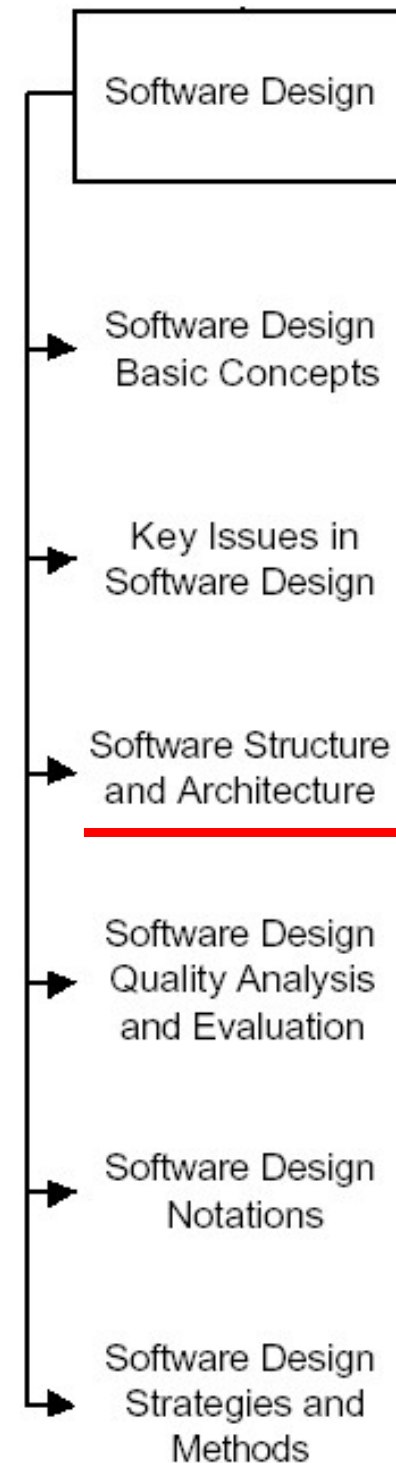
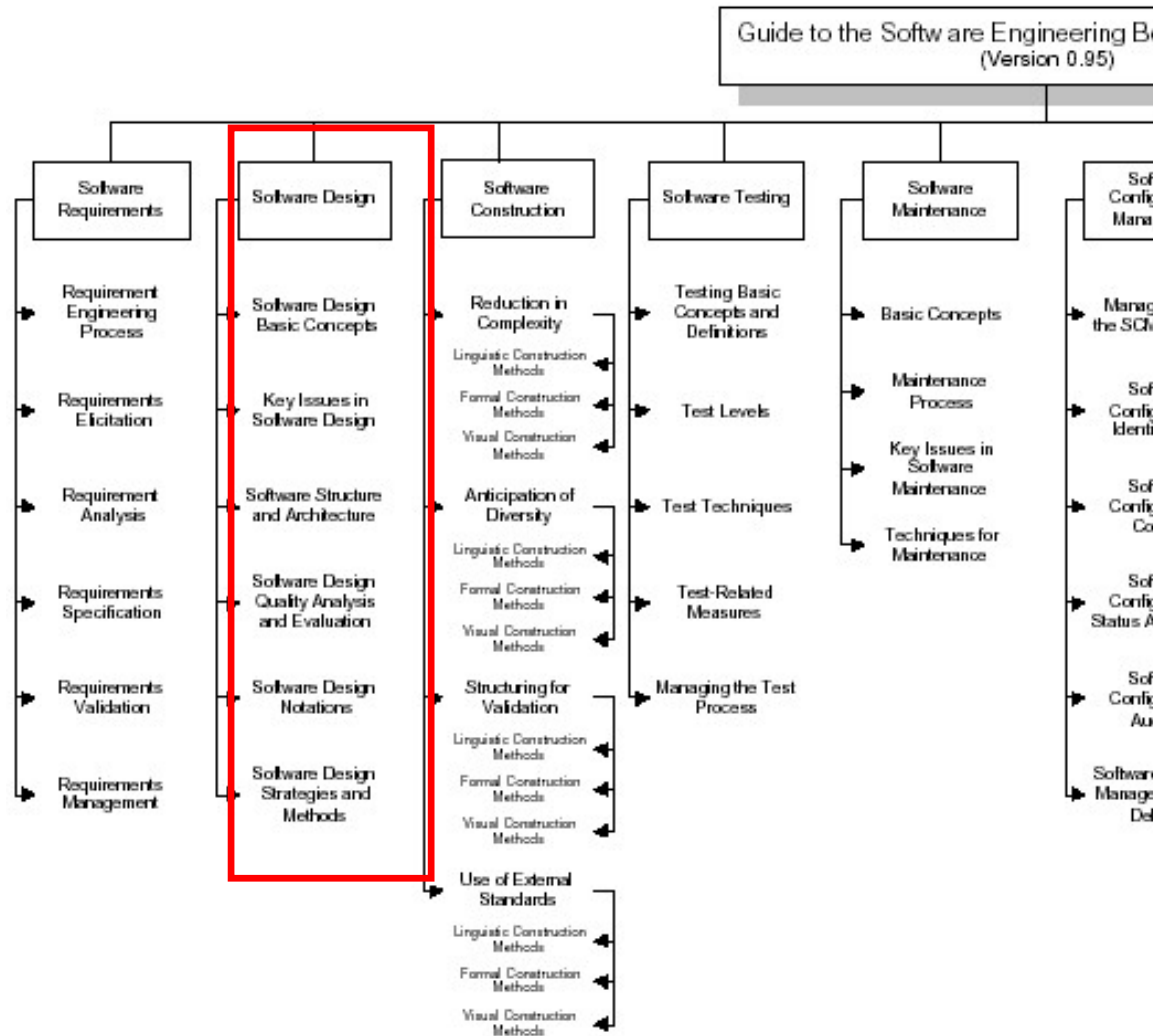
- 组件：具有某种功能的可重用的软件模块单元，表示了系统中主要的计算单元和数据存储。组件有两种：复合组件和原子组件。复合组件由其它复合组件和原子组件通过连接而成。
- 连接件：表示了组件之间的交互，简单的连接件有：管道（pipe）、过程调用（procedure-call）、事件广播（event broadcast）等。复杂的连接件有：客户—服务器（client-server）通信协议，数据库和应用之间SQL连接等。
- 配置：表示了组件和连接件的拓扑逻辑和约束（constraint）。

- 端口：组件作为一个封装的实体，只能通过其接口与外部交互，组件的接口由一组端口组成，每个端口表示了组件和外部环境的交汇点。通过不同的端口类型，一个组件可以提供多重接口。端口可以很简单，如过程调用；也可以很复杂，如通信协议。
- 角色：连接件作为建模软件体系结构的主要实体，同样也有接口，连接件的接口由一组角色组成，连接件的每个角色定义了该连接件表示的交互的参与者。二元连接件有两个角色，如：RPC的角色为caller 和callee， pipe的角色是reading 和writing，消息传递的角色是sender 和receiver等。有的连接件有多于两个的角色，如事件广播有一个事件发布者角色和任意多个事件接收者角色。

好的软件体系结构，
是好的软件的必要条件。

“Good architectural design has always been a major factor in determining the success of a software system.”

3 SA在SE中的地位



Software Structure and Architecture



III. Software Structure and Architecture

Architectural structures and viewpoints

Architectural styles and patterns (macro-architecture)

Design patterns (micro-architecture)

Families of programs and frameworks

4 软件体系结构的理论基础



软件体系结构是软件工程的一个重要的研究和实践领域，也是软件工程的一个新的发展阶段。软件体系结构今天的辉煌离不开软件技术和软件工程几十年来的发展。软件体系结构的理论基础其实有很多在多年以前就已经为人所知。它们主要是在20世纪70年代为结构化程序设计研究而提出的，并伴随着面向对象程序设计和构件等技术的发展而发展。这些理论对于成功地进行软件开发是十分重要的，而对于这种重要性的认识越是当人们感到软件开发困难的时候认识得越深刻。软件体系结构一些重要的理论基础有抽象、封装、数据隐藏、模块化、注意分离点、耦合和内聚、充分性、完备性和简单性、策略和实现的分离、接口和实现的分离、分而制之、层次性。这些理论并不是一个个分离的个体，而是紧密联系在一起的一个整体，共同构成软件体系结构的理论基础。

(1) 抽象

抽象是人们用来处理复杂问题的基本原理之一。抽象有几种形式，如数据抽象、对象抽象、实体抽象、行为抽象、过程抽象等。数据、实体的抽象使得软件操作的对象和参数是针对逻辑结构，而非存储结构。行为、过程的抽象使得操作的指派是依据标识而非地址，由此产生了操作的接口和动态约束描述。在处理系统复杂性方面，抽象起到了重要的作用。减少构件耦合、接口与实现的分离等，都得益于抽象。

(2) 封装

在面向对象程序设计当中，封装是一种重要的机制。所谓对象的概念就是属性（数据）及其操作（行为）的封装体。但封装的机制并不局限于面向对象领域，在结构化程序设计中，封装在函数和子程序当中也得到了体现。封装有利于提高抽象的层次，有利于结构和实现的分离，最终提高了软件的可维护性、可重用性和可靠性。

(3) 信息隐藏

信息隐藏是软件工程的最基本和最重要的原理之一。信息隐藏对用户隐藏了构件的实现细节，用来更好地处理系统的复杂性和减少各构件之间的耦合。为了更好地应用，用户不需要知道的细节都应该由构件封装起来。封装原理经常被用来作为实现信息隐藏的方法，信息隐藏也可以通过接口与实现分离的原理来实现。

(4) 模块化

模块化主要关心的是如何将一个软件系统分解成子系统和构件，其主要任务就是决定怎样将构成应用的逻辑结构物理地分割成代码实体。模块化的主要做法，就是在一个系统内引入具有良好定义的分界，依次来处理系统的复杂性。模块化与封装原理的联系非常密切。模块化带来了复杂系统资源管理、维护 and 应用的逻辑和条理性，增加了应用设计的灵活性。另外，它对于系统的运行设计和管理调度也提供了方便。



(5) 关注分离

不同和无关联的责任应该在软件系统中分离开来，让它们出现在不同的构件中。相互协作完成某一个特定任务的构件应该和其他任务中执行计算的构件分离开来。如果一个构件在不同的环境下扮演不同的角色，在构件中这些角色应该独立且相互分离。例如，在多层体系结构的组件设计中，在多种应用场景中担任不同角色的同一组件，需要并且可以使用不同的接口定义。这样，对某一角色开发的只是与角色相关的信息和服务，避免了过多暴露所造成对应用设计的负担和混乱，并保证了组件运行的可靠和安全。

(6) 耦合和内聚

耦合和内聚最初是作为结构化设计方法的部分原理而提出的。耦合强调模块之间的特性，而内聚强调模块内部的特性。耦合是用来衡量一个模块同另一个模块的联系的紧密程度的。紧密的耦合就会使系统变得复杂，因为如果一个模块和另外的模块有很密切关联的话，这个模块就很难理解、调试、维护。通过弱耦合构件的设计可以降低系统的复杂性。内聚用来衡量单一模块内功能和元素间联系的紧密程度。内聚有几种形式，最期望获得的是功能内聚，它说明一个模块或者构件内的所有元素都共同来完成具有良好边界的行为。最差的形式是偶然内聚，这种形式将毫无联系的抽象放置到同一模块之中。其他形式的内聚还有：偶然内聚、时间内聚、过程内聚、通信内聚、顺序内聚和不规则内聚。

(7) 充分性、完备性和简单性

Grady Booch指出软件系统的每个构件都应该是充分的、完备的和简单的。充分性指的是构件应该把握住与其进行有意义和高效交互抽象的所有特性。完备性指的是一个构件应该把握住所有与其抽象相关的特性。简单性指的是构件所应该完成的操作都可以容易地得到实现。在一个指定的问题寻找构件设计方案时，使其具有充分性和完备性是一个主要目标。

(8) 分而治之

分而治之来自古时的政治，在软件体系结构中该原理得到大量的应用。例如，在设计中将一个复杂任务或构件分割成可以独立设计的更小的部分。该原理经常被用来作为实现注意点分离的方法，但是更重要的还是简化了问题的复杂度。



(9) 策略和实现的分离

软件系统的构件应该实现策略或处理问题，但不能同时处理两者。策略构件负责处理上下文相关的决策、信息的语义和解释的知识、把不相交计算组合成结果、对参数值进行选择等问题。实现构件负责全面规范算法的执行，执行中不需要对上下文相关信息进行决策。由于独立于特定的上下文环境，实现构件容易重用和维护，而策略构件通常是与特定应用相关的，需要随着应用的变化而改变。也可以说策略和实现的分离强调分离出业务逻辑，将业务逻辑部分和与业务逻辑无关的实现部分分离开来，这样软件的结构更清晰。



(10) 接口和实现的分离

任何一个构件都应该包含两个部分：接口和实现。接口部分定义了构件所提供的功能，并规范了功能的使用方法。实现部分包括了实现构件所提供功能的实际代码。该原理的主要目的是防止构件的客户接触到实现的细节，而只为客户提供构件的接口规范和使用方法。另外，该原理还允许独立于其他构件的应用而实现一个构件的功能。就像封装一样，接口和实现的分离也是一种用来获得信息隐藏的技术。接口和实现的分离还支持可修改性，如果构件的接口和实现进行了分离，那么构件它就更容易在系统中进行改变，而且这种分离避免了客户直接受到构件变化的影响。该原理使构件行为和表示的改变变得容易，尤其是那些不影响接口的修改，例如对运行性能的提高。



(11) 层次性

处理复杂问题有两种方法，即：分而制之的横向分割和分层次处理的纵向分割。后者是一种把问题分解成建立在基础概念和思想上多层次的、从底向上逐步抽象的分析和表达的结构，每一层处理该层次的问题、服务于该层次的要求。层次结构在日常生活中是很常见的，例如企业的组织结构和社会结构都是分层的。分层的思想也深入地渗透到人们的思维之中，有些工作比较复杂，我们没法一下子完成，只能把工作分为几层，一层一层地去做，高层工作总是建立在低层工作的基础之上。